GENERATIVE AI ENGINEER

Role-fit interview guide | Chennai position

- Focus: Python, LLM API integration, RAG concepts, backend APIs and cloud workflows.
- Strong evidence: AWS serverless, ETL, Python/SQL, REST APIs and finance automation.
- Gaps to address honestly: LangChain, LangGraph, agentic frameworks, vector DBs.
- Do not claim production ML model training, deployment or research experience.

JD KEYWORDS & FIT

Python • LLMs • RAG • agentic AI

- Strong match: Python, data preparation, REST/API design, AWS orchestration.
- Partial match: LLM integration, prompting, RAG concepts, AI-enabled workflows.
- Unverified / gaps: LangChain, LangGraph, vector databases, agent evaluation.
- Other openings request 5+ years in ML, data science or GCP; do not claim fit.

INTRODUCTION SCRIPTS

60 seconds and 2 minutes

- 60s: “I build Python backend and AWS data workflows, including serverless financial-document automation. I have applied LLM API and prompt automation concepts to workflow use cases and want to build dependable GenAI integrations.”
- 2m: Add an AWS ETL example, explain S3/Lambda/SQS, monitoring and data validation; then explain how Python APIs and RAG concepts transfer to GenAI applications.
- Close: Be specific about what you built versus what you have only studied.

PROJECT DEEP DIVE I

Printsoft replacement • finance ETL

- Printsoft: describe the legacy-document replacement objective, incoming events, SQS/Lambda orchestration, ORDS integration, DynamoDB enrichment and S3 outputs.
- Discuss template selection, naming rules, idempotent reprints, failure recovery and audit trails.
- Finance pipeline: Glue/PySpark, S3, Step Functions, mapping, incremental loading, CloudWatch alerting.
- GenAI transfer: ingestion, retrieval document parsing, data quality and observable APIs.

PROJECT DEEP DIVE II

JazzTrack • AI automation fundamentals

- JazzTrack: Flask REST, MySQL indexing, auth, React UI, workflows and API integration.
- AI automation: explain prompts, LLM APIs, document preparation, RAG concepts without overstating deployment.
- RAG architecture: chunk -> embed -> vector index -> retrieve -> filter -> prompt -> generate -> evaluate.
- ALGOAGENTX (independent): discuss backtesting, data validation, API architecture and risk constraints; do not claim trading returns.

GENAI TECHNICAL Q&A

High-priority fundamentals

- Q: Why RAG over fine-tuning? A: Use retrieval for frequently changing grounded content; tune when behavioral adaptation is the main goal.
- Q: Reduce hallucinations? A: curated retrieval, citations, constrained prompts, confidence/abstain policy, offline evaluation.
- Q: What is LangGraph? A: a stateful graph framework for multi-step agents; mark as a study gap if not used.
- Q: Vector DB? A: stores vectors for approximate nearest-neighbor search with metadata filters; no unsupported production claims.

SYSTEM DESIGN

Secure document-question-answer assistant

- Pipeline: API authentication -> upload -> S3 -> parse -> chunk -> embed -> index -> retrieval -> LLM API -> cited response.
- Apply document-level ACL filtering before retrieval; encrypt data; redact PII in logs.
- Async workers, DLQ, retries, idempotency and monitoring for ingestion failures.
- Quality: retrieval recall@k, grounded-answer accuracy, latency, token costs, prompt-injection testing.

SCENARIOS & TROUBLESHOOTING

Production reliability and controls

- Scenario: duplicate events -> deduplication key, conditional write, retry-safe S3 naming.
- Scenario: model slow -> request tracing, caching, bounded context, async work, timeouts.
- Scenario: wrong answer -> inspect chunks, access filters, embedding/search quality and prompt.
- Scenario: sudden costs -> per-request token/cost tracking, quotas, caching, limits and alerts.

PYTHON / SQL CODING

Whiteboard frameworks

- Python: batch generator for large file input; retry wrapper with exponential backoff; parse nested JSON.
- SQL: detect duplicate invoice IDs with GROUP BY HAVING; latest update via ROW_NUMBER partition.
- PySpark: explain partitions, joins, null handling, skew, incremental loads and data checks.
- Coding drill: implement a small retrieval pipeline with local sample documents, unit tests and error handling.

BEHAVIORAL / FINAL CHECK

STAR answers and readiness

- STAR: discuss an actual automation project, technical constraint, action taken and verified outcome.
- Prepare failure investigation: observe logs, reproduce, isolate, patch, backfill, validate.
- Explain immediate/15-day joining accurately and confirm Chennai 4-days onsite willingness.
- Before sending: verify expected CTC, current location and onsite willingness in email.
- Review Python, RAG, LLM evaluation, LangChain/LangGraph basics, and security.